

Redux E-Commerce Cart: Full Chronological Documentation

This document explains the complete app flow in chronological order, with all key files and logic.

1. Goal of the app

The app has two connected screens rendered together:

- Home: shows products, stock, filters, and cart controls.
- Cart: shows selected items, quantity, totals, and controls.

Main behavior:

- Add to cart decreases product stock.
- Decrease in cart increases product stock.
- Increase in cart decreases product stock.
- Remove item restores full quantity to product stock.
- If quantity reaches 0, item is removed from cart.
- Search by title and category in Home is debounced and stored in Redux.

2. App bootstrap and store setup

2.1 Entry point

File: `src/main.jsx`

What it does:

- Wraps the app with Redux Provider.
- Injects the store into React so all components can dispatch/select.

2.2 Store configuration

File: `src/redux/store.js`

What it does:

- Configures two reducer branches:
 - products -> productReducer
 - cartItems -> cartReducer

Resulting Redux state shape:

- state.products.products
- state.products.filters
- state.cartItems.cartItems
- state.cartItems.totalInCart
- state.cartItems.TotalPrice

3. Initial product loading

3.1 App component

File: src/App.jsx

What it does:

- On mount, simulates async fetch with timeout.
- Dispatches addProducts(productsData) from local JSON.
- Renders Home and Cart side-by-side.

Chronological sequence at app start:

1. App mounts.
2. Simulated fetch completes.
3. addProducts action dispatches.
4. Products are stored in Redux.
5. Home and Cart re-render from Redux data.

4. Products slice (inventory + filters)

File: src/redux/productsSlice.js

State:

- products: array of all products.
- filters:
 - title: current title search text.
 - category: current category filter.

Reducers:

4.1 addProducts

- Replaces products array with loaded data.

4.2 setFilters

- Merges partial filter updates into existing filters.
- Used by debounced Home inputs.

4.3 decrementStock

- Finds product by id.
- Decreases stock by quantity (default 1).
- Uses $\text{Math.max}(0, \text{stock} - \text{qty})$ to prevent negative stock.

4.4 incrementStock

- Finds product by id.
- Increases stock by quantity (default 1).
- Used when item count goes down in cart or item is removed.

5. Cart slice (cart lines + totals)

File: src/redux/cartSlice.js

State:

- cartItems: line items in cart.
- totalInCart: total number of units across all lines.
- TotalPrice: sum of item price x quantity across all lines.

Reducers:

5.1 addToCart

- Finds cart line by product id.
- If exists: quantity + 1.
- Else: adds new line with quantity 1.
- Updates:
 - totalInCart + 1
 - TotalPrice + item.price

5.2 decrementFromCart

- Finds cart line by product id.
- quantity - 1
- Updates:
 - totalInCart - 1
 - TotalPrice - item.price

- If quantity ≤ 0 : removes line from cartItems.

5.3 removeFromCart

- Finds item by id first to know its quantity and price.
- Removes full line from cartItems.
- Updates:
 - $\text{totalInCart} -= \text{removed.quantity}$
 - $\text{TotalPrice} -= \text{removed.price} * \text{removed.quantity}$

Important detail:

- This reducer removes the whole line item.
- Stock restoration for that full line is dispatched separately from UI.

6. Home page flow

File: src/pages/Home.jsx

Data selected from Redux:

- products list
- filters object
- cartItems list

6.1 Debounced filter inputs

Local UI state:

- searchTitle
- searchCategory

Debounce behavior:

1. User types/selects.
2. Local state updates immediately.
3. `useEffect` starts a 400ms timer.
4. After pause, dispatch `setFilters` with title/category.
5. Redux filters update.
6. `filteredProducts` recomputes via `useMemo`.

Filter logic:

- Title match: case-insensitive includes.
- Category match: exact category or all.

6.2 Home cart controls per product

Rendered controls per product:

- Add To Cart button
- - button
- - button
- Quantity display

Rules:

- Add To Cart is disabled when:
 - item is already in cart, or
 - stock is 0.
- - is disabled when stock is 0.

Handlers:

`handleAddToCart(item)`

- Guard `stock > 0`.
- Dispatch `addToCart(item)`.
- Dispatch `decrementStock({ id, quantity: 1 })`.

`handleIncrement(item)`

- Same behavior as add one more unit from Home.
- Dispatch `addToCart` and `decrementStock(1)`.

`handleDecrement(item)`

- Checks if item exists in cart.
- Dispatch `decrementFromCart(item)`.
- Dispatch `incrementStock({ id, quantity: 1 })`.

Result:

- Home immediately reflects cart quantity and live stock.

7. Cart page flow

File: `src/pages/Cart.jsx`

Data selected from Redux:

- cartItems, totalInCart, TotalPrice from state.cartItems
- products from state.products.products

Why products are also selected here:

- To compute availableStock for each cart item.
- To disable + when stock is 0.

7.1 Cart controls per line item

- button

- Dispatch decrementFromCart(item).
- Dispatch incrementStock({ id, quantity: 1 }).

Effect:

- Cart quantity decreases.
- Inventory stock increases by 1.
- If quantity reaches 0, cart line disappears.

+ button

- If availableStock <= 0, no action.
- Dispatch addToCart(item).
- Dispatch decrementStock({ id, quantity: 1 }).
- Button is disabled when availableStock <= 0.

Effect:

- Cart quantity increases.
- Inventory stock decreases by 1.

Remove Item button

- Dispatch removeFromCart(item) to remove full line.
- Dispatch incrementStock({ id, quantity: item.quantity }) to restore full stock.

Effect:

- Full line removed from cart.
- All its units are restored back to product stock.

8. End-to-end chronological scenarios

8.1 First add from Home

1. Click Add To Cart.
2. Cart line created with quantity 1.
3. `totalInCart + 1`, `TotalPrice + price`.
4. Product stock - 1.
5. Add To Cart becomes disabled for that product in Home.
6. Quantity controls stay active.

8.2 Increase quantity with + (Home or Cart)

1. Click +.
2. `addToCart` increments quantity.
3. Totals increase by 1 unit and price.
4. Product stock decreases by 1.
5. If stock becomes 0, + is disabled.

8.3 Decrease quantity with - (Home or Cart)

1. Click -.
2. `decrementFromCart` reduces quantity.
3. Totals decrease by 1 unit and price.
4. Product stock increases by 1.
5. If quantity hits 0, reducer removes cart line.

8.4 Remove full line item

1. Click Remove Item in Cart.
2. `removeFromCart` removes entire line and subtracts full totals.
3. `incrementStock` restores the full removed quantity.

9. Why direct state updates work in reducers

Files involved:

- `src/redux/cartSlice.js`
- `src/redux/productsSlice.js`

You are using Redux Toolkit `createSlice`, which uses Immer.

Meaning:

- Code that looks like mutation is applied to a draft.
- Immer produces a safe immutable next state.

So lines like `existingItem.quantity += 1` are valid in RTK reducers.

10. File map summary

- `src/main.jsx`: Provider + store injection.
- `src/App.jsx`: startup and product loading.
- `src/redux/store.js`: combines slices.
- `src/redux/productsSlice.js`: products inventory + filters.
- `src/redux/cartSlice.js`: cart lines + totals.
- `src/pages/Home.jsx`: product list, debounced filter, Home-side quantity controls.
- `src/pages/Cart.jsx`: cart rendering, totals, cart-side quantity controls and remove.

11. Suggested cleanups (optional)

These are optional improvements for readability and maintainability:

- Rename `TotalPrice` to `totalPrice` for naming consistency.
- Remove `console.log` statements from reducers/components.
- Move duplicated increment/decrement logic into `thunks` for one-action dispatches.
- Add a selector file for derived data (filtered products, cart quantity by id).
- Add tests for reducers:
 - `addToCart` existing vs new
 - `decrementFromCart` quantity to zero
 - `removeFromCart` full-line removal
 - stock sync on `+`, `-`, `remove`
 - filter title/category behavior with debounce timing